Satellite-to-Map Image Translation using Conditional GANs (pix2pix)

Technical Work Report

Author: **Mehdy Mokhtari**
Field: Computer Vision & Generative AI
Date: 2/10/2026

Abstract

This project implements a modular, end-to-end satellite-to-map image translation system using a conditional Generative Adversarial Network (cGAN) based on the pix2pix framework. The model is trained on a paired dataset of approximately 1,500 satellite–map images, which are preprocessed, aligned, and scaled to 128×128 resolution. The architecture employs a U-Net generator and a PatchGAN discriminator, optimized through a composite loss function combining adversarial, L1 reconstruction, and SSIM-based perceptual objectives. The system is designed with production-grade modularity, separating data handling, training, evaluation, and deployment into distinct components. Despite dataset and computational constraints, the final model achieves improved structural consistency and realism, as validated by quantitative metrics (SSIM ≈ 0.65, PSNR ≈ 21.3) and qualitative analysis. This work demonstrates the feasibility of conditional GANs for automated map generation and provides a reproducible, scalable foundation for further development in remote sensing and geospatial visualization.

1. Introduction

1.1 Background

Image-to-image translation is a key task in computer vision, aiming to convert images from one domain to another while preserving spatial and structural information. Translating satellite images into map-style representations is particularly important for applications in remote sensing, GIS, urban planning, and geospatial intelligence, enabling automated map generation and spatial analysis.

1.2 Problem Statement

The goal of this project is to learn a mapping function that converts paired satellite images into corresponding map-style images, preserving spatial structures and visual semantics while ensuring realistic outputs.

1.3 Project Objectives

- Develop a stable conditional GAN (pix2pix) pipeline.
- Preserve spatial and structural consistency in generated maps.
- Evaluate model performance using PSNR and SSIM.
- Maintain a modular, production-ready codebase for reproducibility.

2. Dataset & Preprocessing

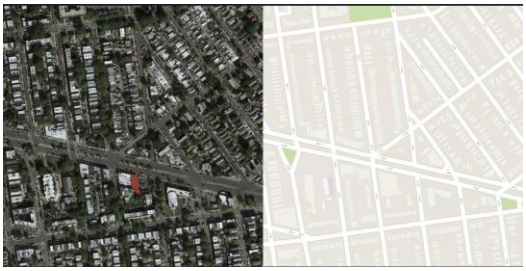

Fig 1. Raw data sample

Fig 2. Processed data sample

2.1 Dataset Description

The dataset used in this project consists of approximately **1,500 paired satellite–map** images designed for supervised image-to-image translation. Each raw sample is provided as a single composite image with a resolution of **1200 × 600 pixels**, where the left half represents the satellite image and the right half represents the corresponding map image.

The raw dataset is organized into initial train and validation directories. During preprocessing, each composite image is split into **two aligned images** (satellite and map), resized, and stored separately while maintaining one-to-one correspondence via **identical filenames**. This pairing strategy ensures precise spatial alignment between inputs and targets, which is critical for conditional GAN training.

All processed images are converted to RGB format with **three channels** and **resized** to a uniform resolution of **128 × 128 pixels**, enabling efficient training and stable convergence.

2.2 Data Splits

The dataset is divided into training, validation, and test sets using the following ratios:

- **70% Training**
- **15% Validation**

- **15% Test**

This split was selected to balance model learning, hyperparameter tuning, and unbiased performance evaluation.

The training set is used to optimize model parameters, the validation set supports monitoring convergence and preventing overfitting, and the test set is reserved exclusively for final evaluation.

The split is performed **deterministically** using a **fixed random seed** to ensure reproducibility across experiments. All splits preserve the paired structure of satellite and map images.

2.3 Dataset Validation

A dedicated validation notebook was created to verify dataset integrity. This included:

- Random paired image visualization
- Shape, channel, and format checks
- Resolution and aspect ratio analysis
- Verification of correct satellite–map alignment

2.4 Preprocessing Pipeline

A dedicated preprocessing pipeline was implemented to transform raw composite images into a structured dataset suitable for conditional GAN training. The preprocessing workflow includes the following steps:

2.3.1 Image Pair Extraction

Each raw image is split horizontally into **satellite** and **map** components, ensuring exact spatial correspondence.

2.3.2 Resizing

Both satellite and map images are resized to **128 × 128 pixels** to standardize input dimensions and reduce computational cost.

2.3.3 Normalization

Images are normalized to the **range [-1, 1]**, matching the generator's **tanh** output activation and improving training stability.

2.3.4 Data Augmentation

During early experiments I tested augmentation and because it gave better results, limited augmentation was applied, including:

- **Horizontal** flips
- **Vertical** flips
- Small random **rotations**
- Minor **brightness** and **contrast** adjustments (satellite images only)

The complete **preprocessing** logic is implemented in
./preprocessing/preprocess_data.py

2.5 Data loading

The processed dataset is accessed through a **custom PyTorch dataset class** that dynamically loads paired satellite and map images, applies normalization, and returns **aligned tensors** suitable for **conditional GAN training**. This modular design ensures clean separation between data handling, model architecture, and training logic, supporting maintainability and scalability.

The **dataset loading** and **batching** are handled via a custom PyTorch Dataset and DataLoader implementation in

./utils/dataloader.py

3. Model Architecture

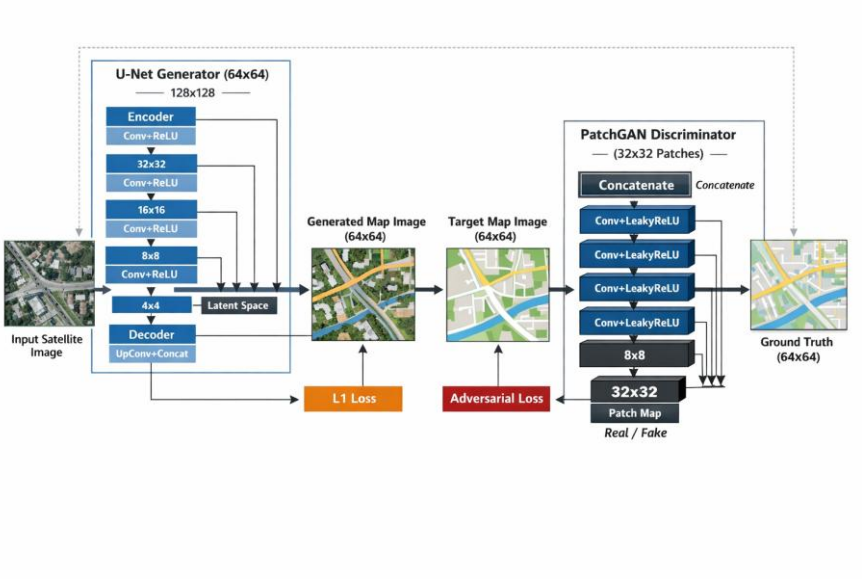

Fig 3. Architecture (not accurate)

3.1 Conditional GAN Overview

A **Conditional GAN** extends the standard GAN formulation by conditioning both the generator and discriminator on an **input image**. In this project, the condition is the satellite image, and the target is the corresponding map-style image.

The **Generator (G)** learns a deterministic mapping

$G : X \rightarrow Y$

where **X** is the **satellite** image and **Y** is the **generated** map image.

The **Discriminator (D)** evaluates whether a given (satellite, map) **image pair** is **real** (ground truth) or **fake** (generated).

The conditioning mechanism is implemented by:

- Feeding the satellite image directly into the generator
- Concatenating the satellite image with either the real or generated map image along the channel dimension before passing it to the discriminator

This formulation enforces input-dependent realism, ensuring that generated outputs are not only visually plausible but also structurally consistent with the input satellite imagery.

3.2 Generator Architecture (U-Net)

The generator is implemented as a **U-Net–based encoder–decoder** architecture specifically adapted for **paired image-to-image translation.** After experimenting with multiple architectural variants and training configurations, a compact U-Net generator was selected due to its training stability, visual quality, and computational efficiency for the satellite-to-map translation task.

Reference implementation: **./models/generator.py**

3.2.1 Architecture Configuration

The final generator configuration is summarized as follows:

- **Architecture**: U-Net
- **Input channels**: 3 (RGB satellite image)
- **Output channels**: 3 (RGB map image)
- Base number of **filters**: 64
- Number of **down-sampling layers**: 4
- Output **activation**: Tanh

This configuration defines a moderate-depth U-Net, which was found to be sufficient for capturing both global layout information and fine-grained spatial details without introducing unnecessary model complexity.

3.2.1 Encoder–Decoder Design

The generator follows a standard **U-Net encoder–decoder** structure:

Encoder (**Downsampling** Path)

The encoder consists of **four** downsampling blocks, where each block reduces the spatial resolution while increasing the number of feature channels starting from a **base width** of **64 filters**. This allows the network to extract increasingly abstract representations of spatial patterns such as road layouts, intersections, and urban regions.

Bottleneck

The bottleneck layer aggregates global contextual information before reconstruction, acting as a **compressed latent representation** of the input satellite image.

Decoder (**Upsampling** Path)

The decoder **mirrors** the encoder structure, progressively **restoring** spatial resolution through upsampling layers and generating the final **map-style output**.

All intermediate layers use **LeakyReLU** activations, while the final output layer applies a Tanh activation function, ensuring compatibility with input images normalized to the range [−1,1].

Skip Connections

A defining characteristic of the U-Net architecture is the use of skip connections between corresponding encoder and decoder layers. These connections allow low-level spatial features—such as edges, boundaries, and linear structures—to bypass the bottleneck and be directly reused during reconstruction.

This design is particularly important for satellite-to-map translation because:

- Fine-grained spatial alignment must be preserved
- Structural elements such as roads and blocks are sensitive to distortion
- Pixel-level correspondence is enforced by the conditional GAN objective

3.3 Discriminator Architecture (PatchGAN)

The discriminator is implemented using a PatchGAN architecture, which evaluates image realism at the patch level rather than the entire image level. This design choice is particularly well-suited for image-to-image translation tasks where local structural correctness is as important as global consistency.

Reference implementation: **./models/discriminator.py**

3.3.1 Architecture Configuration

The final discriminator configuration is defined as follows:

- Architecture: **PatchGAN**
- **Input** channels: 6 (concatenation of satellite image and map image)
- Base number of **filters**: 32
- **Patch size**: 32 × 32
- **Activation** function: LeakyReLU

The discriminator receives a channel-wise concatenation of the input satellite image and either the real or generated map image. This conditional setup allows the discriminator to assess realism in the context of the given satellite image, which is a core principle of conditional GANs.

3.3.2 Patch-Based Discrimination

Instead of producing **a single scalar real/fake** prediction for the entire image, the **PatchGAN** discriminator outputs a **two-dimensional grid** of **predictions**, where each value corresponds to a **local image** patch of **size 32 × 32**.

Each patch-level decision evaluates whether that region of the generated map appears realistic and well-aligned with the corresponding satellite input. This encourages the generator to focus on:

- Local spatial consistency
- Sharp edges and boundaries
- Correct alignment of roads and structural elements
- Uniform realism across the entire image

3.3.3 Patch Size Selection

Multiple PatchGAN configurations were explored during experimentation, including patch sizes of **16 × 16, 32 × 32,** and **64 × 64.**

The **32 × 32** patch size was **selected** as the final configuration because it provided the best trade-off between:

- Capturing **meaningful structural patter**ns such as road continuity
- Avoiding excessive focus on **high-frequency texture** noise
- **Maintaining stable** adversarial training dynamics

Smaller patch sizes tended to **over-emphasize texture**-level details, while **larger patch** sizes behaved more like a **global discriminator** and weakened local supervision.

3.3.4 Activation Function and Feature Scaling

The discriminator employs **LeakyReLU** activations throughout its convolutional layers. This choice improves gradient flow during training and reduces **the risk of**

dead neurons, which is particularly important in adversarial setups where the **discriminator** may otherwise **dominate training early**.

The relatively **lower base filter** count (32 filters) was chosen to keep the discriminator **lightweight** and balanced relative to the **generator**, helping prevent discriminator overpowering and improving overall training stability.

4. Training Pipeline

4.1 Training Strategy

Two training strategies were implemented and compared:

Baseline Training:

The baseline model trains only the generator using supervised learning with an **L1** reconstruction loss. The generator directly learns a mapping from satellite images to map images without adversarial feedback. This setup provides a **stable reference point** and establishes a **lower-bound performance** baseline.

Conditional GAN (pix2pix) Training:

The **main training** pipeline follows a **pix2pix conditional GAN framework**, where the generator (G) and discriminator (D) are **updated** in an alternating manner within **each iteration**.
For each batch:

- The generator produces a map image conditioned on the input satellite image.
- The discriminator is trained to distinguish real satellite–map pairs from generated pairs.
- The generator is then updated to both fool the discriminator and minimize reconstruction errors.

This alternating optimization is essential for conditional GAN training but introduces inherent **stability challenges**, such as **discriminator overpowering** and oscillatory **convergence**. To mitigate these issues, the training pipeline incorporates:

- **Separate learning rates** for G and D
- **Gradient clipping**
- **Balanced loss** weighting
- **Learning rate scheduling** based on validation performance

Training is performed for up to maximum **50 epochs** with **early stopping** based on **validation SSIM**.

Reference: **./training/train_cgan.py** & **./training/train_baseline.py**

4.2 Loss Functions

The generator is optimized using a composite objective function combining adversarial and reconstruction-based losses.

4.2.1 **Adversarial Loss**

A binary cross-entropy (**BCE**) loss is used to encourage the generator to produce map outputs that are **indistinguishable** from **real maps** when conditioned on the satellite image. This loss enforces **local realism** through the **PatchGAN discriminator**.

4.2.1 **L1 Reconstruction Loss**

An L1 loss is applied between the **generated map** and the **ground-truth map** to enforce **pixel-level accuracy** and preserve global spatial structure. This term plays a dominant role in stabilizing training.

4.2.3 **SSIM Loss**

Structural Similarity Index Measure (**SSIM**) is incorporated as a **perceptual loss** to encourage **structural** and **textural** consistency, particularly for roads and spatial layouts. The loss is formulated as 1 – SSIM.

The final generator loss is a weighted sum of these components:

- **Adversarial** loss (weight = 0.5)
- **L1** loss (weight = 200)
- **SSIM** loss (weight = 10)

Careful loss weighting is critical, as it balances realism, spatial alignment, and structural coherence during training.

Reference: **./training/train_cgan.py**

4.3 Configuration Management

All training, model, and loss parameters are managed through a **centralized YAML configuration file**. This design **eliminates hard-coded values** and allows controlled experimentation across different runs.

Using configuration-driven training ensures:

- Full reproducibility of experiments
- Clear tracking of architectural and hyperparameter changes
- Easy comparison between baseline and refined models

This approach was particularly important during the **refinement phase**, where small, systematic changes to learning rates and loss weights were evaluated.

Reference: **./configs/config.yaml**

5. Evaluation & Metrics

5.1 Quantitative Metrics

Model performance is evaluated using **Peak Signal-to-Noise Ratio** (PSNR) and **Structural Similarity Index** (SSIM) on a **held-out test set**.

PSNR measures **pixel-level reconstruction** quality by comparing the generated map to the ground truth. **Higher PSNR values** indicate lower reconstruction error and better overall fidelity. This metric is particularly useful for evaluating global accuracy in image-to-image translation tasks.

SSIM evaluates **perceptual and structural similarity** by comparing luminance, contrast, and spatial structure between images. Since satellite-to-map translation is highly structure-sensitive (e.g., road layouts and boundaries), SSIM is a critical metric for assessing spatial consistency beyond pixel-wise similarity.

For evaluation, the best-performing generator checkpoint (selected based on **validation SSIM**) is loaded, and PSNR and SSIM are computed across the full test set. Final results are reported as dataset-wide averages.

Reference: **./evaluation/metrics.py**

5.2 Qualitative Evaluation

Quantitative metrics are complemented by qualitative visual inspection to assess perceptual quality and failure modes.

Generated outputs are **visualized side-by-side** with the corresponding **satellite inputs** and **ground-truth maps**. This comparison highlights how well the model preserves large-scale structures, road geometry, and spatial alignment.

Qualitative analysis also reveals failure cases, such as:

- **Blurred** or **incomplete** road segments
- Loss of **fine-grained map details**
- **Inconsistencies** in dense or ambiguous regions

This visual evaluation was performed across multiple development phases, including:

- **Baseline** (Phase 4): generator-only outputs used to establish reference quality
- **pix2pix GAN** (Phase 5): improved realism and structural coherence
- **Final evaluation** (Phase 6): systematic comparison, visualization, and reporting

Reference: **./evaluation/visualize.py** , **Phase 4–6 notebooks**

6. Post-processing

A lightweight post-processing step was applied only for visualization purposes to improve the perceptual quality of generated map images. This stage does not affect training or evaluation metrics.

The post-processing operates on the generator output tensors after inference:

- **Output tensors** are converted from the **normalized range [-1, 1] to [0, 255]**
- Images are transformed from **CHW** → **HWC** format and converted to **uint8**
- Optional **sharpening** is applied using unsharp masking via Gaussian blur and weighted addition

Two **sharpening** levels were implemented:

- **Light** sharpening: small kernel (3×3) with mild enhancement
- **Heavy** sharpening: larger kernel (5×5) with stronger edge emphasis

Post-processing is integrated only in the **UI layer** when **displaying generated results**.

All **quantitative evaluations** (PSNR, SSIM) are computed **before post-processing**, using the raw generator outputs to ensure fair and unbiased model assessment.

7. Experiments & Results

This section presents the experimental progression and refinement process carried out.

7.1 Baseline Experiment

Before introducing adversarial training, a **minimal baseline** was established using:

- **Generator-only** U-Net
- **L1** reconstruction loss
- No discriminator

This experiment verified correct **data loading**, **loss computation**, and **metric evaluation**, and provided a **reference PSNR/SSIM** for later GAN-based models.

Result: SSIM ≈ 0.48, PSNR ≈ 18.4

This baseline served as a critical comparison point for all subsequent experiments.

Reference: **./results/logs/baseline/train.log**

7.2 pix2pix GAN Experiments

After validating the pipeline, a full pix2pix cGAN was implemented using a U-Net generator and PatchGAN discriminator. A series of controlled experiments (Experiments 0–14) were conducted, each modifying one major factor at a time.

For better seeing the refinement process you could visit the refinement.log and train.log files which are complete. I have brought a summary of it in the report.

7.2.1 Loss Function Refinement

Adjusting adversarial vs. L1 loss weights showed that over-emphasizing the adversarial loss led to unstable training and degraded SSIM.

Increasing L1 weight and reducing adversarial weight improved structural consistency and training stability.

Adding SSIM directly to the loss yielded improvements.

7.2.2 Architecture Experiments

Increasing generator capacity (base filters 32 → 64) improved optimization smoothness but did not fundamentally improve structure.

Increasing discriminator patch size (16 → 32) led to noticeably better global structure.

Additional generator depth improved visual quality but did not significantly increase SSIM.

7.2.3. Training Dynamics

Learning rate increases did not help escape local minima.

Early stopping based on SSIM prevented overfitting and unstable late-epoch degradation.

Reference: **./results/logs/cgan/train.log** , **./results/logs/refinement.log**

7.3 Resolution scaling (key improvement)

The most significant improvement was achieved by increasing the image resolution:

$64 \times 64 \rightarrow 128 \times 128$

Original learning rates restored. Depth of the architecture only in the decoder section increased.

This change substantially improved both visual quality and quantitative metrics, confirming that resolution was a major limiting factor in earlier experiments.

Result: SSIM ≈ 0.65, PSNR ≈ 21.3

8. System Design & Code Organization

The system is designed with a modular, production-oriented architecture to clearly separate concerns across data handling, model development, experimentation, evaluation, and deployment. This separation improves reproducibility, maintainability, and extensibility, and reflects real-world machine learning system design rather than notebook-centric experimentation.

The overall project structure is shown once below, followed by a functional explanation of each subsystem.

satellite2map_cgan/

├─── models/

├─── training/

├─── evaluation/

├─── preprocessing/

├─── postprocessing/

├─── utils/

├─── ui/

├─── data/

├─── results/

├─── notebooks/

├─── configs/

├─── tests/

8.1 Models

The *models/* directory contains pure **architectural** definitions for the **generator** and **discriminator**. These files are intentionally kept free of training logic, data loading, or evaluation code. This design allows the same model definitions to be reused across baseline training, full GAN training, evaluation, and UI inference without duplication or side effects. **Pretrained weights** are isolated to ensure clean versioning and controlled reuse.

8.2 Training

The *training/* module encapsulates all **learning-time logic**, including optimization, loss composition, and training loops. **Baseline** training (generator-only with L1 loss) is explicitly separated from **full cGAN** training to enforce experimental clarity and to support staged validation of model behavior. This separation prevents tightly coupled scripts and allows controlled comparison between training strategies.

8.3 Evaluation

All **quantitative** and **qualitative** assessment logic is centralized under *evaluation/.* Metric computation (**PSNR**, **SSIM**) and **visualization** are decoupled from training to guarantee that evaluation remains consistent across experiments. This ensures that reported metrics are computed on **raw model outputs** and not influenced by training-time artifacts or UI-specific processing.

8.4 Pre-Processing & Post-Processing

Data preparation (*preprocessing/*) and output enhancement (*postprocessing/*) are treated as independent stages outside the learning pipeline. **Preprocessing** standardizes input format and normalization, while **postprocessing** is used strictly for visualization and user-facing results. Importantly, evaluation metrics are computed before postprocessing, preserving the scientific validity of quantitative results.

8.5 Utilities

The *utils/* directory contains shared infrastructure such as **dataloaders** and **helper** functions. This avoids code duplication across training, evaluation, and UI layers, and enforces consistent data handling throughout the system.

8.6 Supporting Structure

Configuration files, **results**, **notebooks**, and **tests** are isolated into dedicated directories. **Notebooks** are used only for **exploratory analysis** and **reporting**, while the core system remains script- and module-driven. This separation reinforces reproducibility and prevents experimental artifacts from contaminating the main codebase.

9. UI & Deployment

The ui/ module provides an interactive interface (Gradio-based) that consumes trained models as black-box components. It interacts with models and postprocessing utilities but does not perform training or evaluation. This strict boundary ensures that deployment logic does not leak into experimental code, aligning with production ML practices.

In addition to the Gradio demo, a Flask-based web interface is implemented to emulate a realistic deployment setup. The Flask backend serves as a lightweight inference service, exposing trained models through defined routes and treating them strictly as black-box components.

The frontend, built with HTML, CSS, and JavaScript, manages image upload, user interaction, and result visualization, while all inference and postprocessing remain in the backend. No training or evaluation logic is included in the UI layer.

This separation reflects common production ML architectures, ensuring clean boundaries between model execution and presentation logic, and supporting maintainability and future deployment extensions.

10. Limitations & Challenges

I. A **primary limitation** of this project is the **dataset size**, which consists of approximately **1,500 paired satellite–map images**. While sufficient for prototyping and controlled experimentation, this scale limits the model's ability to **generalize** to more diverse geographic patterns and fine-grained structures.

II. **Computational constraints** significantly influenced design decisions. Due to the **lack** of access to a **local GPU** and **restricted** availability of **cloud GPU** resources, most experiments were conducted on limited hardware. As a result, images were initially **downscaled** to **64×64** for rapid iteration and stability testing. Later experiments were extended to **128×128**, which, while an improvement, remains far below the original 600×600 resolution and the **ideal 512×512** commonly used in high-fidelity image translation tasks. This resolution reduction directly impacts sharpness and fine structural detail.

III. Another challenge involved **GAN training instability** and **architectural trade-offs**. Although the **pix2pix framework** is well-documented in literature, **practical constraints** (data scale, resolution, and training time) required deviations from the original setup. Under these conditions, determining how **far performance** could be improved without **overfitting** or **instability** was non-trivial, especially under strict time constraints.

IV. From a modeling perspective, a **persistent sharpness** versus **structural consistency** trade-off was observed. Increasing **adversarial** influence improved visual **sharpness** but risked introducing **artifacts**, while stronger **reconstruction** losses preserved **structure** at the cost of overly **smooth** outputs. Balancing these objectives required careful empirical tuning.

Finally, the project demanded **end-to-end system** engineering, including data pipelines, evaluation tooling, visualization, and UI integration. Ensuring correctness and modularity across all components added complexity beyond model training alone but was necessary to maintain a production-oriented design.

11. Future Work

While the current system demonstrates effective satellite-to-map translation using a pix2pix-style Conditional GAN, several extensions can further improve performance, scalability, and applicability in real-world settings.

First, **expanding** the **training dataset** with larger and more diverse satellite–map pairs would improve generalization across different geographic regions, lighting conditions, and map styles. Increased data diversity is expected to reduce overfitting and improve robustness.

Second, incorporating a **perceptual loss** based on deep feature representations (e.g., **pretrained VGG networks**) could enhance semantic consistency and visual realism. Unlike pixel-wise losses, perceptual losses better capture high-level structural and textural similarities relevant to human perception.

Third, adopting a **multi-scale discriminator** architecture could improve both global coherence and local detail. By evaluating generated images at multiple resolutions, the discriminator can enforce realism across different spatial scales, which is particularly beneficial for complex map structures.

Finally, implementing **tile-based high-resolution inference** would enable the model to scale beyond fixed-size inputs. By processing large satellite images in **overlapping tiles** and seamlessly stitching outputs, the system could support real-world, high-resolution mapping applications.

12. Conclusion

This project implemented an end-to-end **satellite-to-map image translation system** using a **pix2pix-based Conditional GAN**, covering data preprocessing, model design, training, evaluation, and deployment. The system was developed with a modular, production-oriented structure that separates experimentation, evaluation, and user interaction.

The approach works by leveraging paired supervision, adversarial learning, and reconstruction-based losses to preserve spatial structure while translating visual semantics from satellite imagery to map-style representations. Quantitative metrics (PSNR and SSIM) and qualitative analysis confirm that the model learns meaningful and spatially consistent mappings despite data and computational constraints.

This work demonstrates the practical applicability of conditional GANs in **remote sensing and geospatial visualization**, while also highlighting the importance of system design, reproducibility, and engineering discipline in applied machine learning projects. The project provides a strong foundation for future extensions toward higher resolutions, larger datasets, and real-world deployment scenarios.

13. References

[1] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, Image-to-Image Translation with Conditional Adversarial Networks, CVPR, 2017.

[2] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, Image Quality Assessment: From Error Visibility to Structural Similarity, IEEE TIP, 2004.

[3] A. Paszke et al., PyTorch: An Imperative Style, High-Performance Deep Learning Library, NeurIPS, 2019.

[4] M. Abadi et al., TensorFlow: A System for Large-Scale Machine Learning, OSDI, 2016.

[5] OpenCV Contributors, Open Source Computer Vision Library, https://opencv.org/.

[6] F. Pedregosa et al., Scikit-image: Image Processing in Python, JMLR, 2014.

[7] Machine Learning / Deep Learning Course Lecture Notes (PDF), provided by the instructor.